QUALITY ASSURANCE & SOFTWARE TESTING

Advanced Jest Unit & Mutation Testing

Practical Laboratory Comprehensive Submission Report

NO.	STUDENT FULL NAME	STUDENT ID
1	SHAMBEL KIBRE	1601605
2	ABENEZER ANDUALEM	1700018
3	NAOMI DEREJE	1601547

Academic Term: 4th Year, 1st Semester | Quality Assurance (QA)

Submission Date: November 2026 | Automated Test Suite: 12 Suites, 201 Tests Passing (100%)

Executive Summary & Test Suite Results

This report presents the rigorous execution, contract verification, boundary validation, test isolation restructuring, coverage enhancement, and mutation testing analysis for the 14 laboratory questions. The entire project is implemented under Node.js ES Modules with Jest and StrykerJS.

TEST SUITES PASSED	TOTAL TESTS EXECUTED	STATEMENT COVERAGE	BRANCH COVERAGE
12 / 12	**201 / 201**	**97.60%**	**93.56%**

Q#	Topic / Exercise Title	Target Source Module	Test File Location	Status
Q1	Student Registration & Validation	`src/registration.js`	`tests/unit/registration.test.js`	**PASS (14)**
Q2	GradeBook Stateful Classes	`src/grading.js`	`tests/unit/grading.test.js`	**PASS (18)**
Q3	Tuition Fee & Array Processing	`src/tuition.js`	`tests/unit/tuition.test.js`	**PASS (18)**
Q4	Enrollment Management & Side Effects	`src/enrollment.js`	`tests/unit/enrollment.test.js`	**PASS (7)**
Q5	Test Isolation in Student Repository	`src/studentRepository.js`	`tests/unit/studentRepository.test.js`	**PASS (9)**
Q6	Lifecycle Hooks & Resource Cleanup	`src/fakeDatabase.js`	`tests/lifecycle/fakeDatabase.test.js`	**PASS (6)**
Q7	Scholarship Eligibility Decision Table	`src/scholarship.js`	`tests/data-driven/scholarship.test.js`	**PASS (32)**
Q8	Tuition Payment Combinations	`src/payment.js`	`tests/data-driven/payment.test.js`	**PASS (30)**
Q9	Data-Driven Student Result Processing	`src/resultProcessor.js`	`tests/data-driven/resultProcessor.test.js`	**PASS (22)**
Q10	Admission System Coverage Improvement	`src/admission.js`	`tests/coverage/admission.test.js`	**PASS (13)**
Q11	Coverage Thresholds & Regression	`src/courseValidator.js`	`tests/coverage/threshold.test.js`	**PASS (7)**
Q12	First-Order Mutation Testing (Scholarship)	`src/scholarship.js`	`tests/data-driven/scholarship.test.js`	**PASS (Stryker)**
Q13	First- & Higher-Order Mutants	`src/eligibility.js`	`tests/unit/eligibility.test.js`	**PASS (12)**
Q14	Mutation Score Improvement (Payment)	`src/payment.js`	`tests/data-driven/payment.test.js`	**PASS (Stryker)**

Question 1: Student Registration and Validation

> **Mandatory Deliverables:** Establish rigorous registration contract, expose defects with automated tests, refactor validation (positive safe integer ID, nonblank string name, integer age 18-120, documented email format), verify input immutability, compare `toEqual()` vs `toStrictEqual()`, and provide before/after defect log.

1.1 Defect Log Matrix (Before vs. After Refactoring)

ID	Input Tested	Observed Starter Defect	Resolution in Corrected Module
D1	Missing `id` property	Accepted; returned record with `id: undefined`.	Added positive safe integer validation: `Number.isSafeInteger(id) && id > 0`.
D2	`id: -1`, `id: 0`	Accepted non-positive IDs.	Enforced `id > 0` numeric constraint.
D3	`id: 1.5`, `id: "10"`	Accepted floats and numeric strings.	Enforced `typeof id === "number"` and `Number.isSafeInteger`.
D4	`name: 12345`	Accepted numeric name without error.	Added `typeof name === "string" && name.trim().length > 0`.
D5	`age: "20"`	Type coercion in `"20" < 18` allowed string age.	Enforced strict numeric type: `typeof age === "number"`.
D6	`age: 20.5`	Fractional decimal age was accepted.	Added integer constraint: `Number.isInteger(age)`.
D7	`age: 121`	Accepted ages above upper bound.	Added upper boundary constraint: `age <= 120`.
D8	`email: "abc@"`	Accepted malformed email lacking domain.	Replaced `.includes("@")` with standard format regex: `/^[a-zA-Z0-9._%+-]+@(?:[a-zA-Z0-9-]+.)+[a-zA-Z]{2,}$/`.
D9	`null` or non-object	Unhandled TypeError on property access.	Guarded with non-null object type check: `!student || typeof student !== "object"`.

1.2 Immutability and Matcher Comparison (toEqual vs toStrictEqual)

Object Immutability: The function returns a new object reference `{ id, name, age, email, status }`. Deep freezing the input verified that original fields remain unmutated.
Comparison of toEqual() and toStrictEqual(): `toEqual()` checks structural equality but treats missing keys and keys assigned `undefined` as identical. In contrast, `toStrictEqual()` strictly distinguishes between `{ a: 1 }` and `{ a: 1, b: undefined }` and asserts prototype class matches.

Question 2: Student Gradebook and Stateful Classes

> **Mandatory Deliverables:** Validate finite scores from 0 to 100, return null from `getHighestScore()` on empty gradebook, implement `removeLowestScore()` handling ties and single scores, test passing thresholds (49.99, 50, 50.01), and guarantee zero state leakage across test cases.

2.1 Implementation Highlights

- **Score Range Enforcement:** Validates `typeof score === "number" && Number.isFinite(score) && score >= 0 && score <= 100`; rejects `NaN`, `Infinity`, strings, and out-of-range marks.
- **Empty Gradebook Highest Score:** Fixed starter bug where `Math.max(...[])` returned `-Infinity`. Returns `null` when scores array is empty.
- **Lowest Score Removal & Tie Handling:** Finds lowest value with `Math.min`, locates first occurrence via `indexOf`, removes one element via `splice`, and returns the removed number. Returns `null` when empty.
- **Passing Boundary:** `GradeBook.isPassing(average)` evaluates strictly: 49.99 $→$ `false`, 50.00 $→$ `true`, 50.01 $→$ `true`. Fresh instances in `beforeEach()` eliminate cross-test state leakage.

Question 3: Tuition Fee Calculation and Array Processing

> **Mandatory Deliverables:** Validate nonnegative integer creditHours, nonnegative finite pricePerCredit, scholarshipRate within [0, 1] (explicitly rejecting null), round currency to 2 decimal places with stated rounding policy, enforce course immutability, and implement `findMostExpensiveCourse()` with tie-breaking rules.

3.1 Precision Policy and Rounding Formula

Commercial Rounding Policy: Standard round-half-up commercial rounding to two decimal places (cents) is enforced using machine epsilon compensation to eliminate IEEE-754 floating-point drift:

```
const roundCurrency = (amount) => Math.round((amount + Number.EPSILON) * 100) / 100;
```

3.2 Error Hierarchy and Type Contracts

- `courses` must be an array (otherwise throws `TypeError`).
- `scholarshipRate` must be a finite number within [0, 1]. Null explicitly throws `TypeError`; out-of-bounds numbers throw `RangeError`.
- Each course's `creditHours` must be an integer (throws `TypeError`) and nonnegative (throws `RangeError`).
- Each course's `pricePerCredit` must be a finite number (throws `TypeError`) and nonnegative (throws `RangeError`).
- `findMostExpensiveCourse(courses)` calculates total course cost (`creditHours * pricePerCredit`), returning `null` for empty arrays and preserving the first encountered course on ties (via strict `>` comparison).

Question 4: Enrollment Management and Side Effects

> **Mandatory Deliverables:** Mock notifier with `jest.fn()`, verify notification text, parameters, and count, reject duplicate studentId/courseId pairs, implement rollback if notifier throws, and return defensive copies from `getEnrollments()`.

4.1 Architecture and Failure-Handling Policy

```
try {
  if (this.notifier && typeof this.notifier.send === "function") {
    this.notifier.send(studentId, `Enrolled in course \${courseId}`);
  }
} catch (err) {
  // Transactional rollback of newly added record
  const index = this.enrollments.indexOf(record);
  if (index !== -1) this.enrollments.splice(index, 1);
  throw err;
}
```

- **Mock Verification:** `mockNotifier.send` is verified to be called with exact arguments `("S101", "Enrolled in course CS201")` exactly once.

- **Duplicate Rejection:** Second enrollment for identical student and course throws without invoking notifier.
- **Defensive Copying:** `getEnrollments()` returns shallow cloned objects (`.map(r => ({ ...r }))`). Mutating returned items does not corrupt internal state.

Question 5: Test Isolation in a Student Repository

> **Mandatory Deliverables:** Reproduce a test-order failure using one shared repository instance, capture failure evidence, restructure suite with `beforeEach()` and a `createStudentFixture(overrides)` factory, reject duplicate IDs, and restore spies using `afterEach()`.

5.1 Reproduction of Test-Order Failure

When tests share a single module-level instance (`const sharedRepo = new StudentRepository();`), running test B in isolation passes:

```
# Running test B in isolation:
npm test -- tests/unit/studentRepository.test.js -t "test B: expects initial repository to be empty"
Result: PASS (count is 0)

# Running test A and test B sequentially:
npm test -- tests/unit/studentRepository.test.js
Result: FAIL
  Expected: 0
  Received: 1 (polluted by test A's insertion)
```

5.2 Resolution Architecture

- **Isolated Repository Instantiation:** Declared `let repository;` reinitialized inside `beforeEach(() => { repository = new StudentRepository(); });`.
- **Fixture Factory:** Implemented `createStudentFixture(overrides = {})` providing baseline student profiles.
- **Duplicate ID Rejection:** `add(student)` verifies `this.students.some(s => s.id === student.id)` and throws on collisions.
- **Spy Cleanup:** Enforced `afterEach(() => { jest.restoreAllMocks(); });` to prevent spy pollution.

Question 6: Lifecycle Hooks and Resource Cleanup

> **Mandatory Deliverables:** Implement `beforeAll()` connect, `beforeEach()` reset, `afterAll()` disconnect on FakeDatabase, test connected vs disconnected insert, assert nested describe hook execution order via event log, and verify cleanup with disconnect spy.

6.1 Hook Lifecycle Execution Order Log

An event array verified the deterministic nesting of Jest's execution lifecycle:

```
1. nested-beforeAll
2. nested-beforeEach  --> nested-test-1 executes
3. nested-afterEach
4. nested-beforeEach  --> nested-test-2 executes
5. nested-afterEach
6. nested-afterAll
```

Clean disconnection was confirmed by spying on `db.disconnect` in `afterAll()`. Subsequent insertions correctly threw `"Database not connected"` without leaving unhandled failures in the suite.

Question 7: Scholarship Eligibility Decision Table

> **Mandatory Deliverables:** Build at least 20 object-based `test.each()` cases covering all four scholarship outcomes ("Full Scholarship", "Partial Scholarship", "Not Eligible", "Rejected"), boundary thresholds (80/90, 85/95, 12/15), single-condition failures, disciplinary action overrides, type validation, and store fixtures in `tests/fixtures/scholarshipCases.js`.

7.1 Decision Table Rules

- **Rejected:** `disciplinaryAction === true` (preempts all academic achievements).
- **Full Scholarship:** `average >= 90 && attendance >= 95 && credits >= 15 && !disciplinaryAction`.
- **Partial Scholarship:** `average >= 80 && attendance >= 85 && credits >= 12 && !disciplinaryAction`.
- **Not Eligible:** Any student who fails the partial criteria and has no disciplinary action.

7.2 Representative Test Fixture Cases

Case	Average	Attendance	Credits	Disciplinary	Expected Outcome	Status
Full Exact	90.0	95.0%	15	false	Full Scholarship	PASS
Full High	100.0	100.0%	30	false	Full Scholarship	PASS
Full Miss Avg	89.9	95.0%	15	false	Partial Scholarship	PASS
Full Miss Att	90.0	94.9%	15	false	Partial Scholarship	PASS
Full Miss Cr	90.0	95.0%	14	false	Partial Scholarship	PASS
Partial Exact	80.0	85.0%	12	false	Partial Scholarship	PASS
Partial Miss Avg	79.9	85.0%	12	false	Not Eligible	PASS
Partial Miss Att	80.0	84.9%	12	false	Not Eligible	PASS
Partial Miss Cr	80.0	85.0%	11	false	Not Eligible	PASS
Disc Full	98.0	99.0%	20	true	Rejected	PASS
Disc Partial	85.0	90.0%	14	true	Rejected	PASS
Disc Ineligible	40.0	50.0%	5	true	Rejected	PASS

Question 8: Tuition Payment Rules and Data Combinations

> **Mandatory Deliverables:** Use `describe.each()` for card/bank and `test.each()` for domestic/international, create at least 16 invalid/boundary cases, enforce positive finite amount, valid method, boolean international flag, round payment to 2 decimal places, and submit full test matrix.

8.1 Payment Calculation Rules

- **Card:** Fee = $2.5%$ of amount (`amount * 0.025`).
- **Bank:** Fee = Flat $$10.00$.
- **International Surcharge:** Fee += $1.5%$ of amount (`amount * 0.015`) if `isInternational === true` .
- **Rounding:** `Math.round((amount + fee + Number.EPSILON) * 100) / 100` .

8.2 Test-Case Matrix (30 Evaluated Combinations)

Case ID	Amount	Method	International	Expected Outcome	Actual Result	Status
VAL-01	100.00	"card"	false	102.50	102.50	PASS
VAL-02	100.00	"card"	true	104.00	104.00	PASS
VAL-03	250.75	"card"	false	257.02	257.02	PASS
VAL-04	250.75	"card"	true	260.78	260.78	PASS
VAL-05	10,000.00	"card"	false	10250.00	10250.00	PASS
VAL-06	10,000.00	"card"	true	10400.00	10400.00	PASS
VAL-07	100.00	"bank"	false	110.00	110.00	PASS
VAL-08	100.00	"bank"	true	111.50	111.50	PASS
VAL-09	250.75	"bank"	false	260.75	260.75	PASS
VAL-10	250.75	"bank"	true	264.51	264.51	PASS
VAL-11	10,000.00	"bank"	false	10010.00	10010.00	PASS
VAL-12	10,000.00	"bank"	true	10160.00	10160.00	PASS
RND-01	33.33	"card"	false	34.16	34.16	PASS
INV-01	0	"card"	false	Throws "Invalid amount"	Threw expected Error	PASS
INV-02	-100	"card"	false	Throws "Invalid amount"	Threw expected Error	PASS
INV-04	NaN	"card"	false	Throws "Invalid amount"	Threw expected Error	PASS
INV-05	Infinity	"bank"	false	Throws "Invalid amount"	Threw expected Error	PASS

Case ID	Amount	Method	International	Expected Outcome	Actual Result	Status
INV-10	100.00	"paypal"	false	Throws "Unsupported payment method"	Threw expected Error	PASS
INV-15	100.00	"card"	"true"	Throws "Invalid international flag"	Threw expected Error	PASS

Question 9: Data-Driven Student Result Processing

> **Mandatory Deliverables:** Create at least 15 parameterized cases (empty, 0/100, duplicates, decimals, boundaries below/at 50), test strings/nested arrays/NaN/Infinity as invalid inputs, refactor to enforce numeric range [0, 100], verify input immutability, and submit result table.

9.1 Parameterized Result Table (22 Evaluated Scenarios)

Case ID	Input Scores	Expected Result	Actual Result	Status
TC-01	`[]`	{ average: 0, status: "No Scores" }	{ average: 0, status: "No Scores" }	PASS
TC-02	`[0]`	{ average: 0, status: "Fail" }	{ average: 0, status: "Fail" }	PASS
TC-03	`[100]`	{ average: 100, status: "Pass" }	{ average: 100, status: "Pass" }	PASS
TC-04	`[0, 100]`	{ average: 50, status: "Pass" }	{ average: 50, status: "Pass" }	PASS
TC-05	`[50]`	{ average: 50, status: "Pass" }	{ average: 50, status: "Pass" }	PASS
TC-06	`[49.99]`	{ average: 49.99, status: "Fail" }	{ average: 49.99, status: "Fail" }	PASS
TC-07	`[50.01]`	{ average: 50.01, status: "Pass" }	{ average: 50.01, status: "Pass" }	PASS
TC-08	`[49.9, 49.9]`	{ average: 49.9, status: "Fail" }	{ average: 49.9, status: "Fail" }	PASS
TC-11	`[35.5, 64.5]`	{ average: 50, status: "Pass" }	{ average: 50, status: "Pass" }	PASS
TC-13	`[75.25, 85.75, 90]`	{ average: 83.67, status: "Pass" }	{ average: 83.67, status: "Pass" }	PASS
ERR-01	`null`	Throws TypeError	Threw TypeError	PASS
ERR-03	`[80, "90", 75]`	Throws TypeError	Threw TypeError	PASS
ERR-05	`[NaN]`	Throws TypeError	Threw TypeError	PASS
ERR-07	`[-0.01]`	Throws RangeError	Threw RangeError	PASS
ERR-09	`[100.01]`	Throws RangeError	Threw RangeError	PASS

Question 10: Admission System Coverage Improvement

> **Mandatory Deliverables:** Record baseline coverage from 2 tests, add targeted tests for all return paths and boundaries (age 17/18, score 59/60, 74/75, 89/90), target 100% statements/branches, provide before/after metrics table, and explain an admission-rule combination not guaranteed by branch coverage alone.

10.1 Before vs. After Code Coverage Metrics

Metric	Initial Baseline (2 Tests)	Final Targeted Suite (13 Tests)	Measured Improvement
Statements	100.00% (16/16)	100.00% (16/16)	Maintained Complete (0.00%)
Branches	33.33% (4/12)	100.00% (12/12)	**+66.67%**
Functions	100.00% (1/1)	100.00% (1/1)	Maintained Complete (0.00%)
Lines	100.00% (16/16)	100.00% (16/16)	Maintained Complete (0.00%)

10.2 Analysis: Condition Combination Not Guaranteed by Branch Coverage Alone

Short-Circuit Evaluation & Condition Masking: In `if (isInternational && score < 75)`, branch coverage requires only that the compound decision evaluates to `true` and `false` at least once. However, when `isInternational` is `false`, short-circuit evaluation prevents the second operand (`score < 75`) from being evaluated. A test suite can achieve 100% branch coverage with a domestic applicant having score 70, without ever verifying what happens when `isInternational: true` and `score >= 75`. Full combinatorial testing (MC/DC) is required to guarantee that international students with high scores properly bypass review.

Question 11: Coverage Thresholds and Regression Testing

> **Mandatory Deliverables:** Configure global thresholds (90% statements, 85% branches, 90% functions, 90% lines), introduce a new exported function with untested branches causing a coverage regression, restore thresholds by writing tests, compare global vs per-file metrics, and explain why coverage alone is insufficient via a weak assertion.

11.1 Controlled Regression and Recovery Workflow

- **Thresholds Configured:** `{ statements: 90, branches: 85, functions: 90, lines: 90 }` in `jest.config.js`.
- **Controlled Regression:** Added `src/courseValidator.js` with multiple untested branches (length < 5, department mismatch). Coverage on functions dropped to exactly 90.00% and per-file coverage on `courseValidator.js` sat at 0.00%.
- **Recovery:** Added `tests/coverage/threshold.test.js` exercising all 6 branches of course validation. All metrics restored: Statements 97.55%, Branches 97.14%, Functions 93.33%, Lines 97.55%.

11.2 The Oracle Problem (Weak Assertion Demonstration)

```
// High execution coverage achieved, but zero verification of correctness:
test("weak assertion example", () => {
  const result = validateCourseCode("CS101", "CS");
```

```
  expect(result).toBeDefined(); // WEAK ORACLE
});
```

Why Coverage Alone Is Insufficient: Code coverage only measures what machine instructions were executed by the CPU. If a defect inverted `valid: true` to `valid: false`, the test still passes because `{ valid: false }` is defined. High coverage without rigorous assertions creates a false sense of security.

Question 12: First-Order Mutation Testing of Scholarship Rules

> **Mandatory Deliverables:** Run StrykerJS targeting `src/scholarship.js`, save killed/survived/timeout counts, select at least 5 first-order mutants from the report, add targeted tests killing non-equivalent mutants, rerun mutation testing, and submit a 5-row mutant analysis table.

12.1 StrykerJS Quantitative Mutation Results

- **Total Mutants Generated:** 107
- **Initial Run:** Killed: 103 | Survived: 4 | Timeout: 0 | Score: **96.26%**
- **Post-Improvement Run:** Killed: 104 | Survived: 3 | Timeout: 0 | Score: **97.20%**
- **Adjusted Mutation Score: 100.00%** (after mathematically justifying 3 equivalent mutants).

12.2 Five-Row Mutant Analysis Table

ID	Mutation Injected	Initial Status	New Targeted Test Designed	Final Status
M1 (9)	`TypeError("...")` $→$ `TypeError("")`	**Survived**	Asserted exact error message string: `.toThrow("Student must be a valid object")`.	**Killed**
M2 (73)	`return "Rejected"` $→$ `return ""`	**Killed**	Disciplinary action boundary test expecting exact non-empty return string `"Rejected"`.	**Killed**
M3 (80)	`average >= 90` $→$ `average > 90`	**Killed**	Exact boundary input `average: 90` expecting `"Full Scholarship"`.	**Killed**
M4 (76)	`&& credits >= 15` $→$ `|| credits >= 15`	**Killed**	Low average and attendance with high credits (20) expecting `"Not Eligible"`.	**Killed**
M5 (67)	`Error("Disciplinary...")` $→$ `Error("")`	**Killed**	Asserted exact error string for non-boolean disciplinary action flag.	**Killed**

Question 13: First-Order and Higher-Order Mutant Investigation

> **Mandatory Deliverables:** Baseline tests for `checkEligibility()`, implement 3 first-order mutants (FOM1, FOM2, FOM3), 1 second-order mutant (SOM), and 1 third-order mutant (TOM), verify killings using `test.each()`, and explain interactions and fault coupling.

13.1 Mutant Distinguishing & Killing Matrix

Mutant	Applied Mutation	Killing Input `(age, score, doc)`	Original Out	Mutant Out	Status
FOM1	`age >= 18` $→$ `age > 18`	`(18, 70, true)`	"Eligible"	"Not Eligible"	**KILLED**
FOM2	`score >= 60` $→$ `score > 60`	`(20, 60, true)`	"Eligible"	"Not Eligible"	**KILLED**
FOM3	First `&&` $→$ `||`	`(16, 70, true)`	"Not Eligible"	"Eligible"	**KILLED**
SOM	Both relational: `age > 18 && score > 60`	`(18, 60, true)`	"Eligible"	"Not Eligible"	**KILLED**
TOM	All 3: `(age > 18 || score > 60) && doc`	`(16, 75, true)`	"Not Eligible"	"Eligible"	**KILLED**

13.2 Higher-Order Interactions and Masking

Fault Masking in TOM: Relational mutations (`>`) make the conditions strictly narrower, whereas the logical mutation (`||`) makes the condition broader. At input `(18, 70, true)`, the test that killed FOM1 is masked in TOM because `(18 > 18 || 70 > 60) = (false || true) = true`! Thus, higher-order mutants demonstrate that combining syntactic changes can create coupled defects that escape first-order tests.

Question 14: Mutation Score Improvement Project

> **Mandatory Deliverables:** Configure StrykerJS to mutate `src/payment.js`, classify surviving mutants across fee arithmetic, international logic, and errors, add at least 8 targeted Jest tests, calculate mutation score via formula, compare with Stryker, and justify any equivalent mutants.

14.1 Eight Targeted Improvement Tests

- **Targeted Test 1:** Half-cent floating point tie rounding (40.20 card domestic $→$ 41.21).
- **Targeted Test 2:** Minimum positive transaction boundary (0.01 bank $→$ 10.01).
- **Targeted Test 3:** Exact error string assertion: `"Invalid amount: must be a positive finite number"`.
- **Targeted Test 4:** Exact error string assertion: `"Unsupported payment method: must be 'card' or 'bank'"`.
- **Targeted Test 5:** Exact error string assertion: `"Invalid international flag: must be a boolean"`.
- **Targeted Test 6:** Bank international surcharge fee arithmetic verification (200 bank intl $→$ 213.00).
- **Targeted Test 7:** Card fee percentage calculation without precision drift (80 card $→$ 82.00).
- **Targeted Test 8:** Combined 4.0% fee rate on international card payment (500 card intl $→$ 520.00).

14.2 Quantitative Results & Formula Scoring

Formula: `Mutation Score = (Killed + Timeout) / (Total Mutants) * 100%`

- **Total Mutants:** 52
- **Killed:** 49
- **Timeout:** 0
- **Survived:** 3 (all 3 justified as mathematically equivalent)
- **Raw Mutation Score:** `49 / 52 * 100% = 94.23%`
- **Adjusted Mutation Score:** `49 / (52 - 3) * 100% = 100.00%`

14.3 Technical Justification for Equivalent Mutants

1. **Line 13 (Guard Subsumption):** `typeof amount !== "number" || !Number.isFinite(amount)` mutated to `false || !Number.isFinite(amount)`. In JavaScript, `Number.isFinite(x)` is false for all non-numbers. Replacing the type check with false cannot alter execution on any input.
2. **Line 33 (Exhaustive Branching):** `else if (method === "bank")` mutated to `else if (true)`. Because preceding validation already rejected any method other than "card" or "bank", if method is not "card", it is guaranteed to be "bank". Mutating this guard to true is an equivalent mutant.
3. **Line 41 (Machine Epsilon Invariant):** `(amount + fee + Number.EPSILON)` mutated to `- Number.EPSILON`. At transaction scales multiplied by 100 (> 100), $2.22 × 10^{-16}$ is below the IEEE-754 double precision least significant bit, rendering $+ epsilon$ and $- epsilon$ bit-identical under floating-point arithmetic.

> **Verification Statement:** All 14 laboratory exercises have been completely executed, tested, and verified with 100% passing automated test suites and high-fidelity mutation scores.